

⇒ LSTMP
RNN

⇒ (Long Short Term
Memory RNN)

⇒ Simple RNN → Long Term Dependencies
Vanishing gradient problem

⇒ Problems with RNN:
(see Previous Section of RNN)

Task ⇒ Predict Next word in Sentence.

* The Color of The Sky is Blue.

This word is
dependent of Two words

"Color & Sky" for prediction.

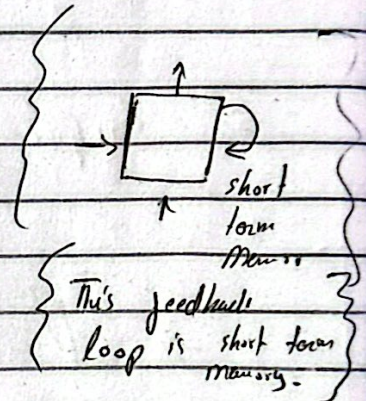
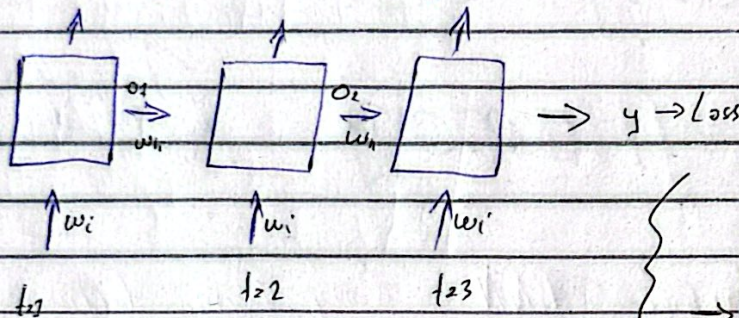
* & gap b/w These two words is less, so we won't
see vanishing gradient problem.

Another Sentence:

I grew up in Pakistan I speak
fluent .
 ↓

⇒ This word's prediction depends on "Pakistan" &
There is huge gap b/w them.
⇒ Long Term dependency & will face
vanishing gradient problem.

⇒ Basic Representation of RNN



⇒ LSTM RNN:

⇒ LSTM RNN
 ↙ Short term memory
 ↘ Long term memory

→ LSTM RNN has Both Long & short
term memory.

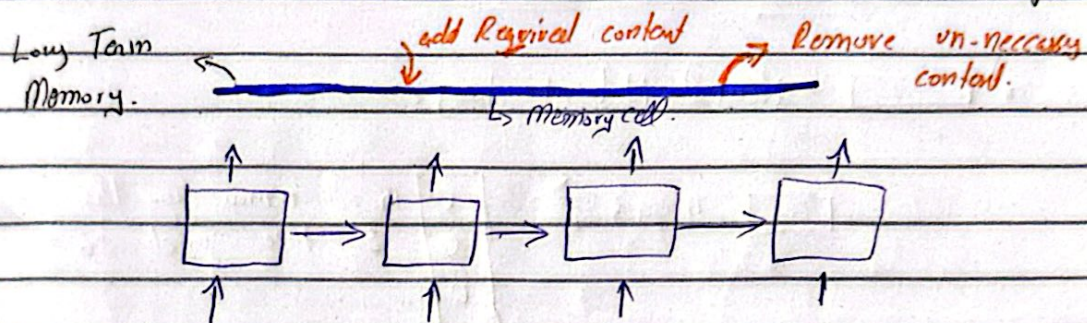
★ Feedback loop of RNN is short term
memory.

⇒ Conveyance Belt : (in Airport for Luggages).

can place luggages
→ can Take out luggages.

* Similarity in LSTM There will only line for long term memory.

→ Main work ⇒ Add what content is required removed what content is not required.



LSTM RNN Architecture

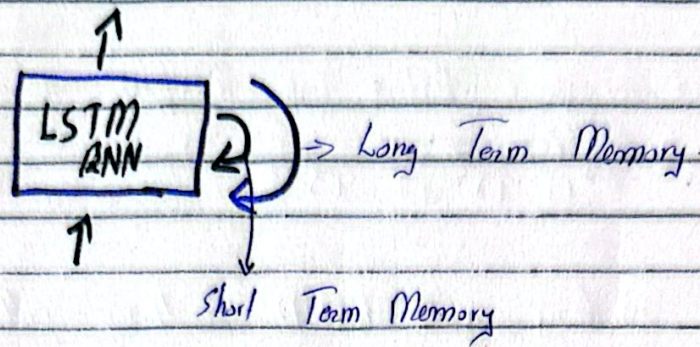
* Three imp gates.

① Forget gate

② Input gate & Candidate Memory

③ Output gate

*** A single Hidden Layer neuron has these three gates.



$Y \text{ --- } X$

Basic Notations

-  $\rightarrow$ Neural Network Layer

 means $\Rightarrow$ $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$ in all neuron in this layer
I have used "G" function

- $\odot \rightarrow$ Pointwise wise operation

$\{ \otimes \rightarrow \text{Pointwise Multiplication}, \oplus \rightarrow \text{pointwise addition} \}$

$$\begin{aligned} V_1 &= \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \\ V_2 &= \begin{bmatrix} 4 & 5 & 6 \end{bmatrix} \end{aligned}$$

* Applying operation like this is called point wise
it can be $\times$ tion, $+$ tion etc.

$$\begin{aligned} V_1 &= \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \\ V_2 &= \begin{bmatrix} 4 & 5 & 6 \end{bmatrix} \end{aligned} \quad \otimes \Rightarrow \begin{bmatrix} 4 & 10 & 18 \end{bmatrix}$$

$$\left\{ \begin{aligned} 1 \times 4 &= 4 \\ 2 \times 5 &= 10 \\ 3 \times 6 &= 18 \end{aligned} \right\}$$

- $\rightarrow$ Vector Transfer

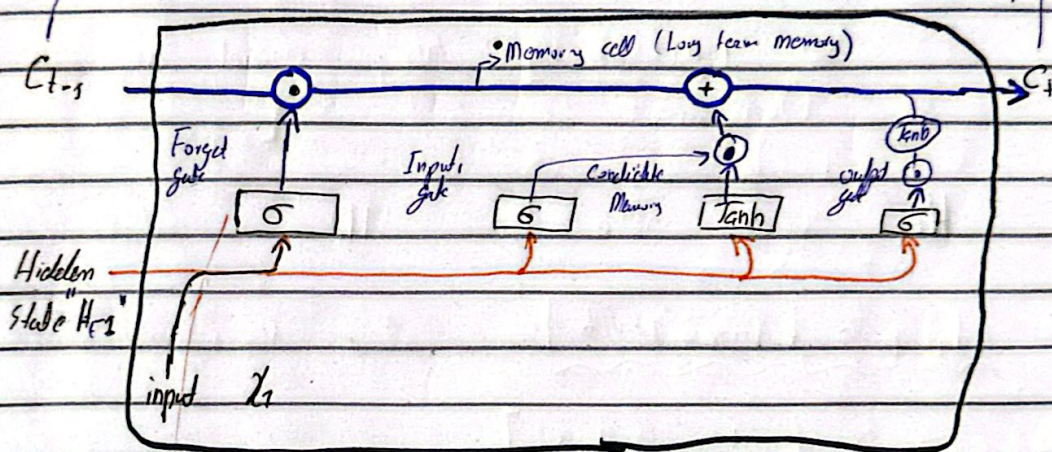
- $\searrow$ Concatenate (combine two vectors "not add")

$$\begin{matrix} V_1 = [1 & 2 & 3] \\ V_2 = [4 & 6 & 8] \end{matrix} \searrow \rightarrow [1 & 2 & 3 & 4 & 6 & 8]$$

- $\swarrow$ Copy (Make copy of a Vector)

Memory cell
of previous time stamp $\times$

- After we remove & add some content
- state of memory cell become ct. for that time stamp



- $h_{t-1} \Rightarrow$ Hidden State of previous time stamp

- $x_t \Rightarrow$ Word processed as input in current time stamp.

- $C_{t-1} \Rightarrow$ Memory Cell of previous time Stamp

- $C_t \Rightarrow$ Memory Cell of Current time Stamp after removing or adding some content

↳ Forget Gate:

⇒ I have Text & I need to predict next word.

Text	Next Word
x_{11} x_{12} x_{13} x_{14}	y_{15}

↳ I have converted These words into Vector

$$x_{11} = [0 \ 2 \ 4 \ 6], x_{12} = [45 \ 12] \dots$$

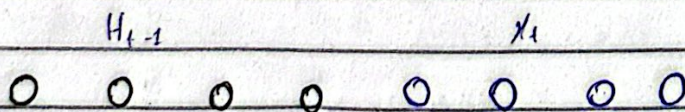
⇒ I am representing every word in "4 Dimensions":

* H_{t-1} will also be 4D & C_{t-1} too

$$\Rightarrow H_{t-1} \Rightarrow [1 \ 2 \ 4 \ 6]$$

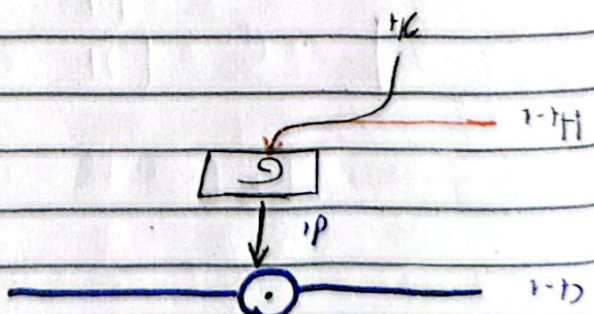
$$\Rightarrow C_{t-1} \Rightarrow [4 \ 6 \ 1 \ 5]$$

*** See previous graph/diagram H_{t-1} & x_t Concatenates (combines) & becomes the input.



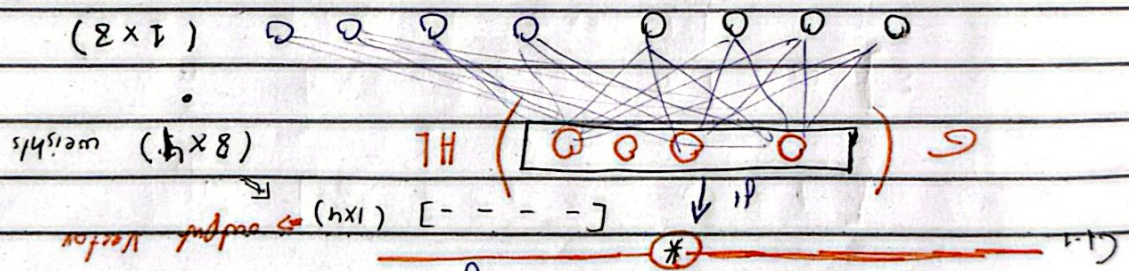
at $t=2$ I am passing this info, Combined

$$d_t = \sigma(w_t \cdot [h_{t-1}, x_t] + b)$$



Now the combined input gets into a neural network.

e.g. in hidden layer we have 4 neuron & all have "σ" function.



$$\Rightarrow (1 \times 8) \cdot (8 \times 3) \Rightarrow (1 \times 4)$$

e.g. in previous state we got $d_{t-1} = \begin{bmatrix} 6 & 8 & 12 & 6 \end{bmatrix}$

we $d_t = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}$ (output)

So, $C_{t-1} \otimes d_t$

$$\begin{bmatrix} 6 & 8 & 12 & 6 \\ 0 & 0 & 0 & 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Removing all the previous context

② 2nd Scenario:

$$if \ I \ gate \ f_t = [1 \ 1 \ 1 \ 1]$$

$$Then \begin{bmatrix} 6 & 8 & 12 & 16 \\ 1 & 1 & 1 & 1 \end{bmatrix} \Rightarrow \odot \Rightarrow \begin{bmatrix} 6 & 8 & 12 & 16 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

kept all the previous context.

③ 3rd Scenario:

$$f_t = [0.5 \ 1 \ 0.5 \ 1]$$

$$Then \begin{bmatrix} 6 & 8 & 12 & 16 \\ 0.5 & 1 & 0.5 & 1 \end{bmatrix} \Rightarrow \odot \Rightarrow \begin{bmatrix} 3 & 8 & 6 & 16 \end{bmatrix}$$

- I have removed some context from these two vectors
- kept the other two same.

X — X

Here I am removing information either full or some (Not adding info, just keeping it the same).

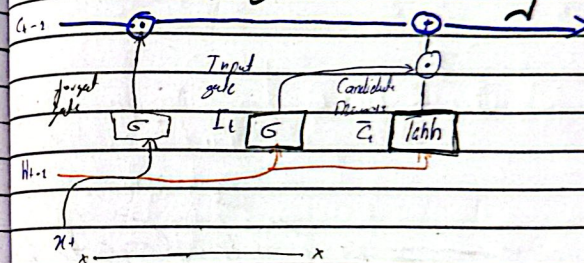
As I am removing info so $\rightarrow$ Forget Gate

Conclusion: Based on context $\Rightarrow$ gate will go some or will not go some information.

X — X

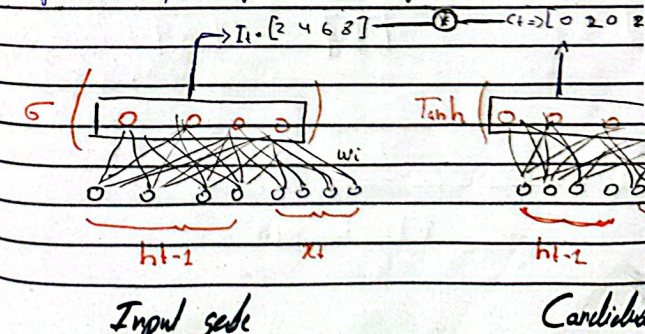
Input Gate &

Candidate Memory: (Ha In)



e.g we get $I_t = [2 \ 4 \ 6 \ 8]$

If Any information needs to be added in memory after this point operation $\rightarrow$ that information will be added

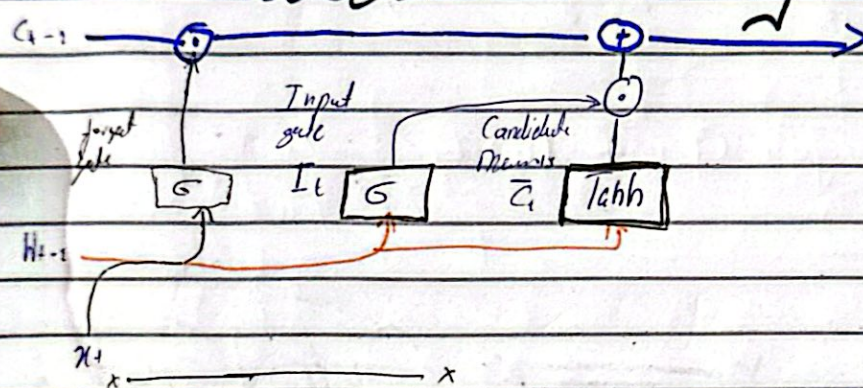


⇒ Conclusion: Based on Context → Forget
 or will not let go some information.

$x \text{ --- } x$

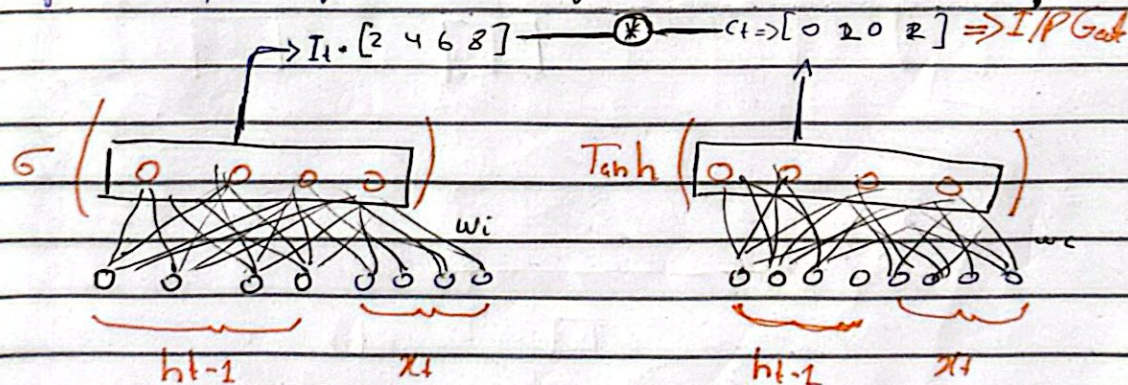
⇒ Input Gate &

Candidate Memory: (Adding Information)



e.g we get $I_t = [2 \ 4 \ 6 \ 8]$

*** If Any information needs to be added in memory C_{t-2}
 after this point operation → that information will be added.



Input gate

Candidate Memory

$$\Rightarrow I_t = \sigma(w_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\Rightarrow C_t = \tanh(w_c \cdot [h_{t-1}, x_t] + b_c)$$

eg

$$I_t = [2 \ 4 \ 6 \ 8] \Rightarrow \odot \Rightarrow [0 \ 8 \ 0 \ 16]$$

$$C_t = [0 \ 2 \ 0 \ 2]$$

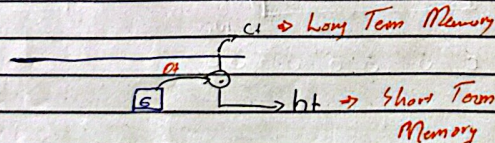
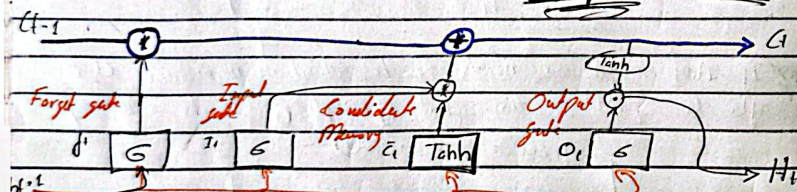
* We got this info & we add it. & we get C_t

$\Rightarrow$ Forget gate + Input gate & Candidate Memory.

$$C_t = f_t * C_{t-1} + I_t * C_t$$

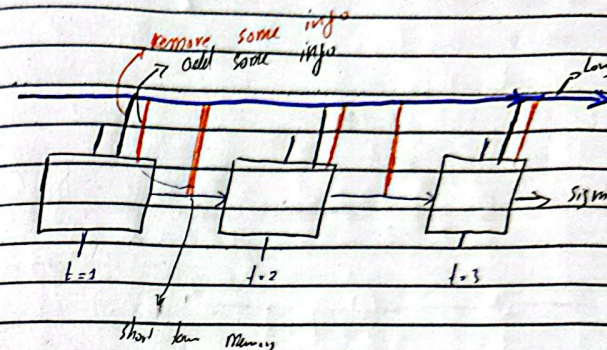
• See graph you'll get this equation.

Output Gate:



$$\Rightarrow O_t = \sigma(w_o \cdot [h_{t-1}, x_t] + b_o)$$

$$\Rightarrow h_t = O_t * \tanh(C_t)$$



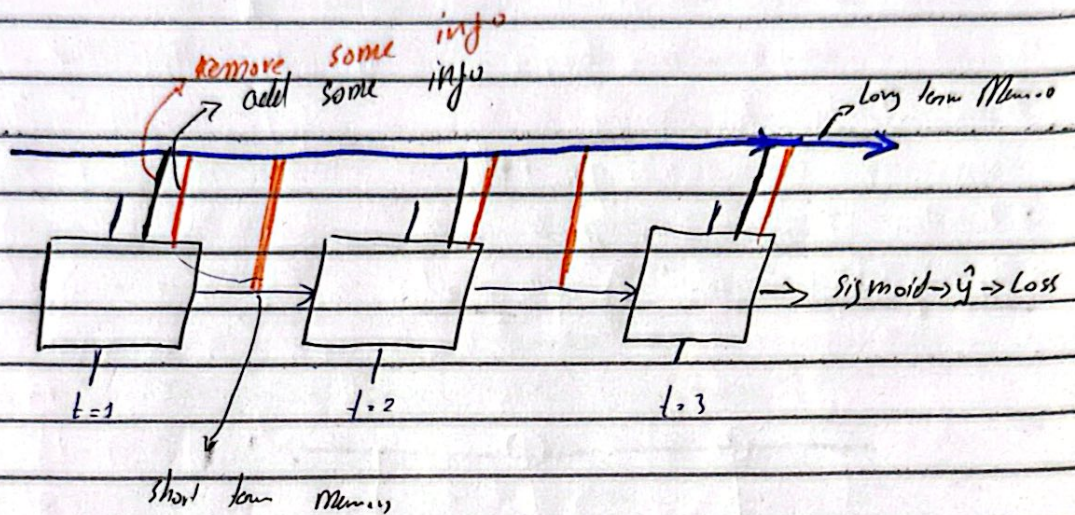
Weights That needs to Update.

$$[w_i, w_c, w_o]$$

$$x \text{ --- } x$$

$$\Rightarrow O_t = \sigma(w_o \cdot [h_{t-1}, x_t], b_o)$$

$$\Rightarrow h_t = O_t * \tanh(c_t)$$



Weights That needs to Update.

$$[w_i, w_c, w_o]$$

x ————— x

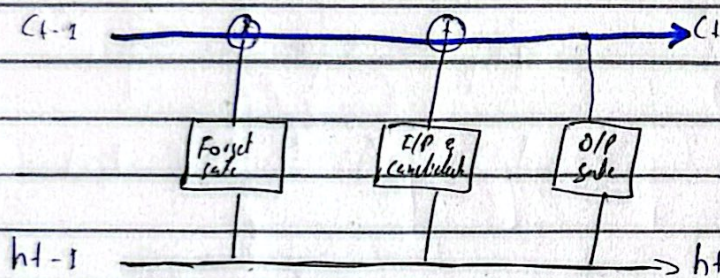
⇒ Training Data with LSTM RNN.

Text Paragraph

O/P [good/bad]

I went to Restaurant &
ordered a burger, The burger
looked Tasty & crispy, but
burger is not good for health,
but this burger was made with
why protein so it was good!

1



⇒ Word ⇒ Vectors ⇒ Embedding Layer.

• We use Word2Vec & Convert every word into 3D vector

• 3D Vector →

	[Good]	[Bad]	[Healthy]
Words Tasty	0.9	0.0	0.1

⇒ These vector will keep on changing ⇒

e.g. When "Burger is not good for health" is parsed

then good decreases & Bad increases & healthy
also decrease ⇒ [0.6 0.7 0.0]

"Burger is made with why protein" again change ⇒ [0.8 0.3 0.4]

e.g. We have 3D vector about good, bad, healthy
 $[\text{good} \quad \text{bad} \quad \text{healthy}]$ in Longterm Memory.

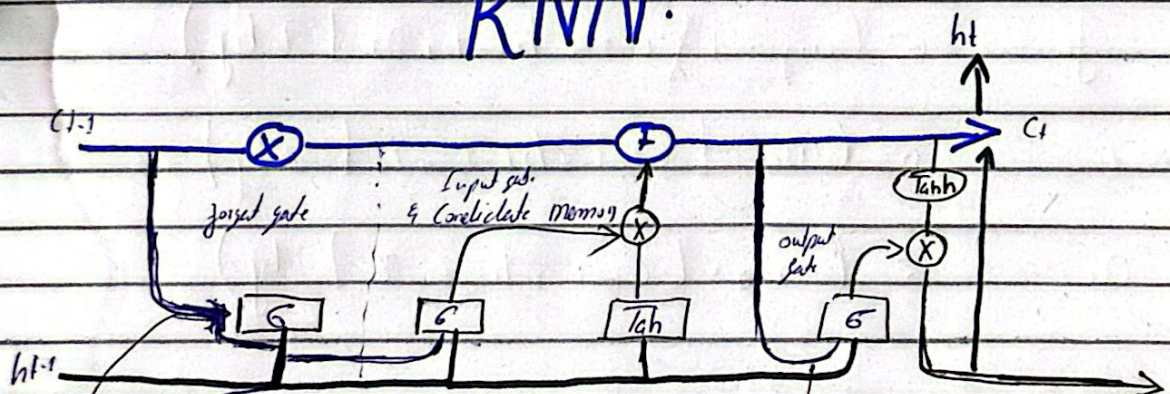
→ We change its values as new sentences are
 passed → We will forget some info &
 add some some info.

changes as new sentences pass

initially $\Rightarrow [0.0 \quad 0.2 \quad 0.4] \xrightarrow{\uparrow} [0.4 \quad 0.1 \quad 0.3]$
 $[0.7 \quad 0.4 \quad 0.8] \leftarrow$

x ————— x

⇒ Variants of LSTM RNN.



✗ All things same but There is this additional line that we send to "forget & input gate" & also to "output gate".

→ This additional line

** These Connections From Memory Cell
 to forget gate
 input gate $\Rightarrow$ peephole Connections.
 output gate

Peephole Connections: Through These Connections.
 we let the gate layers look at
 cell state.

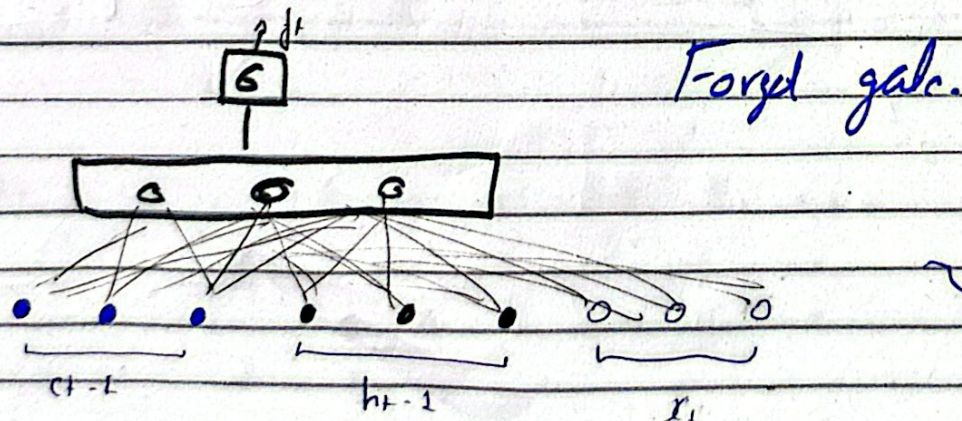
eg Forget gate $\Rightarrow$ if you want to make memory
 cell forget something from cells, Take
 previous info from memory cell to
 decide what info it need to forget.

* Look at cell state Then make a decision.

$$f_t = \sigma(w_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(w_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$O_t = \sigma(w_o \cdot [C_{t-1}, h_{t-1}, x_t] + b_o)$$

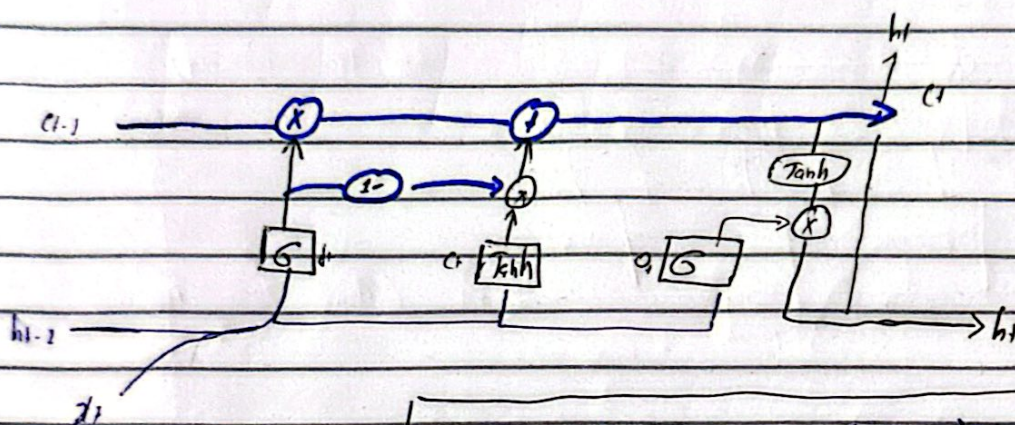


* Everything same just added (C_{i-1})

x ——— x

Another Variation.

⇒ Coupling Forget & Input gates.



$$\Rightarrow \boxed{* C_i = f_i * C_{i-1} + (1 - f_i) * C_i}$$

* Instead of Separately Deciding what to forget & what new info. we make This Decision Together.

* A Input & Forget gate is Combined.

Goal: * We only Forget when we are going to input something in its place.

* We only input new values when we are forgetting something.